Title: Model Driven Mixed Reality Tours for Museums.

A model-driven engineering pipeline that turns early interaction designs for museum tours into runnable AR prototypes. Designers specify visitors, real artifacts, adapters (trackers/beacons), and system feedback at a platform-independent level, then auto-map to a Unity/AR Foundation.

Objective

Deliver a model-driven pipeline that turns early mixed-interaction designs for museum tours into a runnable AR app (Unity/AR Foundation) with traceability from design decisions to code and content.

Models

What models and metamodels are needed?
I'll use four metamodels:
MuseumMM (PIM)
Rooms, zones, paths, artifacts (id, label, pose), constraints (no-touch, quiet area).

TourMM (PIM)
Visitor types (adult/kid), tour stops, narratives, learning goals, triggers (arrive/scan/tap).

AdapterMM (PIM)
Adapters = inputs (image markers, BLE, QR) and outputs (visual overlays, audio, haptics); events, payloads, and feedback mappings. Adapters explicitly model the "bridge" between real objects and digital content.

UnityMM (PSM)
Unity/AR Foundation elements: Scenes, Prefabs, GameObjects, Components (.ARTrackedImage, ARAnchor, AudioSource, Canvas), and simple Script stubs.

How many domains, MMs, and their relations.
Domains (3):
• Museum domain (MuseumMM)
• Mixed-interaction design domain (TourMM + AdapterMM)
• Runtime platform domain (UnityMM)

Relations:
MuseumMM + TourMM + AdapterMM → M2M → a merged Deployment model (still PIM) → M2M → UnityMM (PSM).
UnityMM then drives code/assets generation (M2T).

Tool integration - external tools that produce or consume models. model compliance

Consumes: Unity/AR Foundation consumes UnityMM via generated scenes/prefabs/scripts (M2T).

Produces: An ImageLibrary.json (from AdapterMM) for AR tracked images; a BeaconMap.csv (from AdapterMM/MuseumMM) for BLE regions.

Compliance: PIMs defined in EMF/Ecore; transformations in ATL/QVTo; code gen in Acceleo

How will the model be created, updated?
MuseumMM: initially manual (import CSV of artifacts).

TourMM: manual by designer (personas, stops, narratives).

AdapterMM: manual for inputs/outputs; derived defaults suggested (“artifact with QR ⇒ tracked image input + 2D overlay feedback”).

UnityMM: generated from the PIMs; designers can regenerate when PIM changes (simple “listeners”: change in MuseumMM/TourMM triggers M2M+M2T run).

Transformations

What transformation will you develop?

- (M2M) PIM→PIM
  MuseumMM + TourMM + AdapterMM → DeploymentPIM
- (M2M) PIM→PSM
  DeploymentPIM → UnityMM
- (M2T) PSM→Code
- (M2T)PIM→Tool Artifacts

How will you combine transformations?
M2M (compose PIMs) → M2M (to UnityMM) → M2T (scenes/prefabs/C#) → open in Unity. M2T (AdapterMM → ImageLibrary/BeaconMap) can run anytime; Unity picks them up if present